

Multiway Number Partitioning en qutrits:

Qudit-Adapt

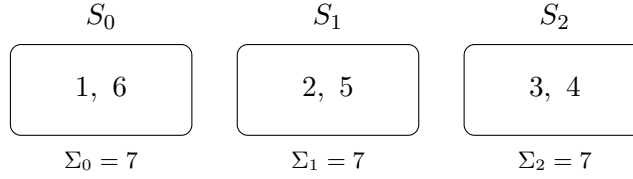
1. El problema

Dado un conjunto de N números positivos $\{a_1, \dots, a_N\}$, el problema de *multiway number partitioning* (o *k-way partitioning*) consiste en repartirlos en k subconjuntos disjuntos $S_1, \dots, S_k$ tales que las sumas parciales

$$\Sigma_i = \sum_{j: a_j \in S_i} a_j, \quad i = 1, \dots, k$$

sean lo más parejas posible entre sí. Es NP-hard para todo $k \geq 2$ fijo (el caso $k = 2$ es el *number partitioning* clásico, NP-complete en su versión de decisión [2]).

Ejemplo. Para $S = \{1, 2, 3, 4, 5, 6\}$ y $k = 3$, la partición $S_0 = \{1, 6\}$, $S_1 = \{2, 5\}$, $S_2 = \{3, 4\}$ es óptima: la suma total es $21 = 3 \times 7$, y esta asignación la reparte exactamente parejo, $\Sigma_0 = \Sigma_1 = \Sigma_2 = 7$.



Por qué qudits y no qubits. Con qubits, una etiqueta de k valores necesita *one-hot* (k qubits por elemento más una penalización para forzar exactamente un 1). Con un *qudit* de dimensión $d = k$ alcanza un solo qudit por elemento, sin restricción auxiliar: su estado *es* la etiqueta.

2. Planteamiento del problema

2.1. Variables y notación

Sea $z = (z_1, \dots, z_N)$ con $z_j \in \{0, 1, 2\}$ la etiqueta (parte) asignada al elemento j , y

$$\Sigma_i(z) = \sum_{j=1}^N a_j \delta_{i, z_j}, \quad i = 0, 1, 2,$$

la suma acumulada de la parte i .

2.2. Costo de referencia: Deller et al.

La función de costo típica en la literatura, propuesta por Deller et al. [1] (Apéndice B.2) para multiway number partitioning, penaliza la diferencia entre cada par de partes:

$$C_{\text{Deller}}(z) = \sum_{a < b} [\Sigma_a(z) - \Sigma_b(z)]^2.$$

El cuadrado –en vez del valor absoluto– hace que el costo sea siempre ≥ 0 , valga 0 exactamente en balance perfecto, y sea diferenciable, lo cual importa para el optimizador clásico dentro del ADAPT.

2.3. Equivalencia con la desviación respecto del promedio

Como cada elemento a_j cae en exactamente una parte, la suma de las tres $\Sigma_i(z)$ es siempre el total del conjunto, sin importar z :

$$\Sigma_0(z) + \Sigma_1(z) + \Sigma_2(z) = \sum_{j=1}^N a_j =: 3\mu, \quad \forall z.$$

Es decir, $\mu = \frac{1}{3} \sum_j a_j$ es también el promedio de las tres sumas parciales, para cualquier reparto. Expandiendo C_{Deller} :

$$\begin{aligned} C_{\text{Deller}}(z) &= \sum_{a < b} (\Sigma_a - \Sigma_b)^2 = 2 \sum_i \Sigma_i^2 - 2 \sum_{a < b} \Sigma_a \Sigma_b \\ &= 2 \sum_i \Sigma_i^2 - \left[\left(\sum_i \Sigma_i \right)^2 - \sum_i \Sigma_i^2 \right] = 3 \sum_i \Sigma_i^2 - (3\mu)^2, \end{aligned}$$

donde en el segundo renglón usamos $(\sum_i \Sigma_i)^2 = \sum_i \Sigma_i^2 + 2 \sum_{a < b} \Sigma_a \Sigma_b$ y luego $\sum_i \Sigma_i = 3\mu$. Por otro lado,

$$3 \sum_i (\Sigma_i - \mu)^2 = 3 \sum_i \Sigma_i^2 - 6\mu \sum_i \Sigma_i + 9\mu^2 = 3 \sum_i \Sigma_i^2 - 18\mu^2 + 9\mu^2 = 3 \sum_i \Sigma_i^2 - 9\mu^2,$$

idéntico al resultado anterior. Concluimos:

$$C_{\text{Deller}}(z) = 3 \sum_{i=0}^2 [\Sigma_i(z) - \mu]^2$$

misma física, mismo argmín, solo cambia la normalización de energía por un factor 3.

2.4. El problema a resolver

Usamos la forma sin el factor 3 porque generaliza sin cambios notacionales a más de 3 partes y porque es la misma estructura que ya usa el resto del proyecto (penalización cuadrática tipo Burges, igual que `Hp_factorizacion`):

$$z^* = \underset{z \in \{0,1,2\}^N}{\operatorname{argmin}} C(z), \quad C(z) = \sum_{i=0}^2 [\Sigma_i(z) - \mu]^2.$$

3. La codificación

3.1. Operador dígito por sitio

Reciclamos la convención de `funciones/utilidades_factorizacion.py`: para el sitio (qu-trit) j definimos el **operador dígito**

$$\hat{d}_j = \hat{J}_z^{(j)} + \hat{I}, \quad \text{autovalores } \{0, 1, 2\},$$

que ya existe en el código como `digit_operator(n, site)`. El autovalor de $\hat{d}_j$ en un estado de la base computacional *es* la etiqueta (parte) a la que quedó asignado el elemento j .

3.2. proyectores de etiqueta vía interpolación de Lagrange

Para promover $\Sigma_i(z)$ a operador necesitamos, para cada parte $i \in \{0, 1, 2\}$, el **proyector** $\hat{\Pi}_i^{(j)}$ que vale 1 si el elemento j tiene etiqueta i y 0 en caso contrario -es decir, la versión operatorial de δ_{i,z_j} . Como $\hat{d}_j$ tiene espectro finito $\{0, 1, 2\}$, cualquier función de sus autovalores –en particular la función indicadora– es exactamente un polinomio de grado ≤ 2 en $\hat{d}_j$, dado por el polinomio interpolador de Lagrange sobre esos tres puntos:

$$\hat{\Pi}_i(\hat{d}) = \prod_{m \neq i} \frac{\hat{d} - m \hat{I}}{i - m}.$$

Explícitamente:

$$\hat{\Pi}_0(\hat{d}) = \frac{1}{2}(\hat{d} - \hat{I})(\hat{d} - 2\hat{I}) = \frac{1}{2}\hat{d}^2 - \frac{3}{2}\hat{d} + \hat{I}, \quad (1)$$

$$\hat{\Pi}_1(\hat{d}) = -\hat{d}(\hat{d} - 2\hat{I}) = -\hat{d}^2 + 2\hat{d}, \quad (2)$$

$$\hat{\Pi}_2(\hat{d}) = \frac{1}{2}\hat{d}(\hat{d} - \hat{I}) = \frac{1}{2}\hat{d}^2 - \frac{1}{2}\hat{d}. \quad (3)$$

3.3. Hamiltoniano de costo

Con $a_1, \dots, a_N$ los números a repartir, promovemos $\Sigma_i(z)$ a operador diagonal

$$\hat{\Sigma}_i = \sum_{j=1}^N a_j \hat{\Pi}_i(\hat{d}_j), \quad i = 0, 1, 2,$$

y promovemos $C(z)$ de la §2 al Hamiltoniano de costo:

$$\boxed{H_p = \sum_{i=0}^2 (\hat{\Sigma}_i - \mu \hat{I})^2} \quad \mu = \frac{1}{3} \sum_j a_j.$$

Referencias

- [1] Y. Deller, S. Schmitt, M. Lewenstein, S. Lenk, M. Federer, F. Jendrzejewski, P. Hauke, V. Kasper, *Quantum approximate optimization algorithm for qudit systems*, arXiv:2204.00340 (2023). Apéndice B.2 (Multiway number partitioning).
- [2] S. Mertens, *The Easiest Hard Problem: Number Partitioning*, arXiv:cond-mat/0310317 (2003).